

Laporan Pembuatan Gateway “Babel” Tower of Babel



Oleh:

Muchammad Rafif Azis Syahlevi - 18224123

**LABORATORIUM SISTEM TERINTEGRASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA -
KOMPUTASI**

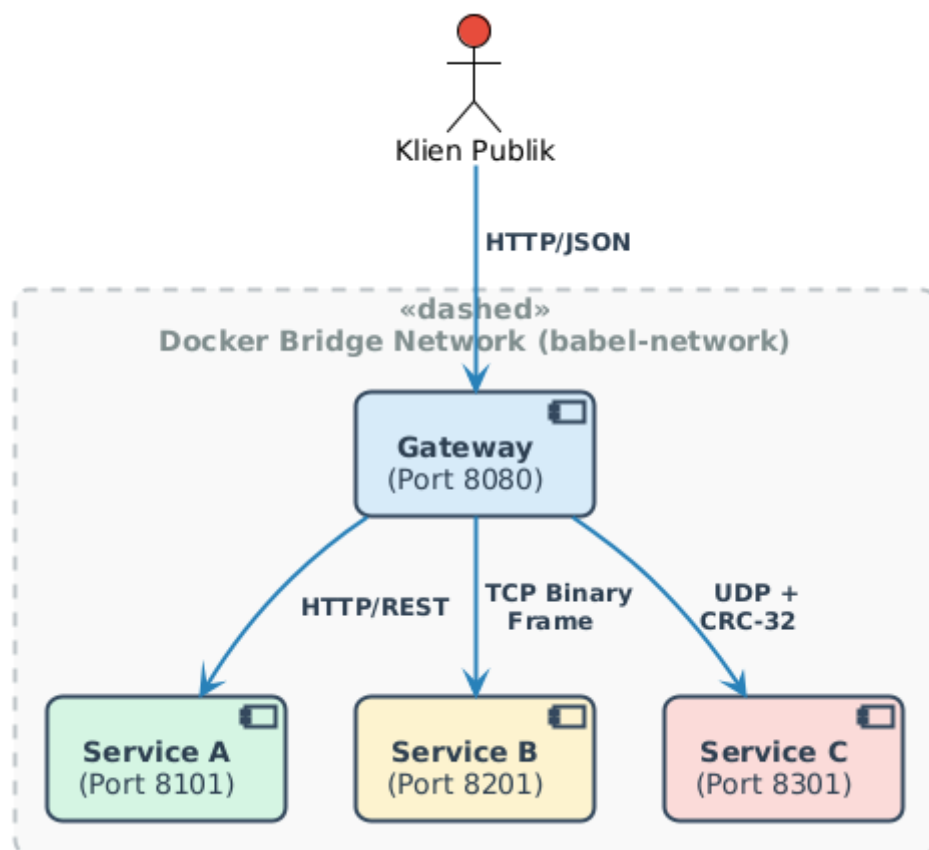
INSTITUT TEKNOLOGI BANDUNG

2026

1. Arsitektur Gateway

Babel Gateway dirancang sebagai *middleware* asinkron berkinerja tinggi menggunakan *framework* FastAPI dan server ASGI Uvicorn berbasis Python `asyncio`. Secara arsitektural, sistem ini menerapkan pola desain *Adapter* dan *Facade* secara bersamaan. Pola *Facade* menyederhanakan kompleksitas interaksi dengan menyajikan satu antarmuka REST (HTTP/JSON) yang seragam bagi klien eksternal. Sementara itu, pola *Adapter* bekerja di belakang layar untuk menjembatani dan menerjemahkan payload klien ke dalam ragam protokol spesifik server *backend* (HTTP, TCP, UDP).

Seluruh ekosistem layanan di orkestrasi secara terpusat menggunakan Docker Compose di atas jaringan internal babel-network berjenis *bridge*. Keputusan topologi ini sangat krusial untuk aspek keamanan dan *network isolation*, server *backend* sepenuhnya tertutup dari dunia luar dan tidak mengekspos *port* publik apa pun. Klien eksternal dipaksa untuk mendelegasikan seluruh panggilannya melalui Gateway sebagai *Single Point of Contact* pada *port* 8080.



Gambar 1. Diagram Arsitektur

2. Service Registry

Gateway mengimplementasikan *Service Registry* berbasis *In-Memory Static Configuration*. Metadata fungsional untuk seluruh server disimpan dalam struktur data *Dictionary* (*SERVICES*) pada memori saat Gateway melakukan *startup* proses inisialisasi awal. Informasi yang disimpan mencakup konfigurasi infrastruktur seperti *host*, *port*, tipe protokol dan daftar kapabilitas operasi logis yang didukung oleh masing-masing *backend* misalnya: pemetaan bahwa *Service A* mendukung *uppercase*, sedangkan *Service C* mendukung *sum*.

Pendekatan *in-memory* ini dipilih secara sadar untuk menghindari tambahan *network hop* atau dependensi pada pangkalan data eksternal seperti Redis di level *middleware*. Hal ini memastikan bahwa proses penelusuran (*lookup latency*) rute dan kapabilitas dieksekusi murni pada RAM, menjamin kompleksitas waktu $O(1)$ waktu konstan yang sangat esensial untuk meminimalisasi latensi intrinsik Gateway.

3. Strategi Translasi Protokol

Translasi protokol adalah urat nadi operasional dari Gateway ini. Karena sistem dituntut untuk berinteraksi dengan layanan tingkat rendah, saya menggunakan pustaka bawaan *struct* pada Python untuk presisi manipulasi *byte*, *endianness* spesifiknya *Big-Endian*, dan *padding*.

1. Service A (HTTP-JSON): Gateway bertindak sebagai *Reverse Proxy* asinkron menggunakan modul *httpx.AsyncClient*. Gateway memetakan ID permintaan logis klien ke dalam *header X-Request-ID* sesuai kontrak. Penggunaan *AsyncClient* memastikan bahwa koneksi HTTP dapat digunakan kembali, sehingga menghemat *overhead* pembuatan soket pada setiap permintaan.
2. Service B (TCP-Frame-JSON): Gateway mengatasi masalah klasik jaringan TCP, yaitu tidak adanya batas pesan yang jelas atau disebut *TCP stream boundaries loss*, dengan menyuntikkan *header* 16-byte (format struct: >H B B I Q). *Header* ini memuat *Magic Word* (0xBABE) sebagai identifier protokol untuk menolak aliran acak, informasi versi, dan panjang *payload* 4-byte *integer*. Kehadiran informasi panjang *payload* memberi instruksi absolut kepada server kapan siklus pembacaan aliran byte harus dihentikan.
3. Service C (UDP-CRC-JSON): Komunikasi *connectionless* ini ditangani menggunakan abstraksi tingkat rendah *asyncio.DatagramProtocol*. Karena UDP tidak menjamin sampainya paket, Gateway merakit *header* 20-byte komprehensif (format struct: >H B B I Q B B H), termasuk *Sequence Number* internal dan pemetaan *Opcode* biner misal: 0xC0DE. Untuk menjamin integritas paket terhadap gangguan jaringan, Gateway menghitung nilai *checksum* validasi menggunakan algoritma *zlib.crc32* dari keseluruhan paket berisi *header* + *payload* dan melampirkannya sebagai 4-byte *unsigned integer* di ujung paling akhir paket.

Tabel 1. Pemetaan Biner

Byte 0-1	Byte 2	Byte 3	Byte 4-7	Byte 8-15	Byte Selanjutnya
Magic (0xBABE)	Version (1)	Status (0)	Payload Length	Request ID	JSON String (UTF-8)

4. Strategi Routing

Fungsi rute utama pada *endpoint /execute* beroperasi secara dinamis dengan mengevaluasi nilai *req.operation* klien terhadap metadata kapabilitas di dalam *registry* internal. Terdapat dua fase pemrosesan rute:

- Preferred Service Selection: Jika klien secara eksplisit menyertakan *hint preferred_service* pada opsi permintaan, dan layanan target tersebut mendukung operasi yang diminta, Gateway akan menghormati preferensi tersebut dan mengarahkan lalu lintas ke sana. Hal ini memberikan fleksibilitas kepada klien untuk melakukan *A/B testing* atau menargetkan *backend* spesifik.
- Deterministic Fallback: Jika parameter *preferred service* kosong atau target tidak mendukung operasi terkait, Gateway akan menjalankan strategi *fallback* dengan cara mengiterasi *registry* secara sekuensial dan menyerahkan *request* ke *backend* kompatibel pertama yang ditemukan. Lebih lanjut, operasi yang sama sekali tidak didukung contoh: *request* dengan *opcode* fiktif akan dievaluasi dan langsung ditolak dengan kode HTTP 400/503 di level API Gateway, sebuah mekanisme pertahanan penting untuk mencegah *backend* terbebani oleh *junk requests*.

5. Model Concurrency

Untuk memenuhi ekspektasi lalu lintas tinggi, sistem ini direkayasa secara murni *Non-Blocking*. Model *concurrency* digerakkan oleh *Event Loop* dari *asyncio* dan *Uvicorn*. Konsep ini memungkinkan *Gateway* untuk melakukan perpindahan konteks secara efisien, ketika Gateway sedang menunggu respons jaringan I/O dari Service B (TCP) yang lambat, *thread* utama tidak akan pernah terkunci. Sebaliknya, *Event Loop* akan melanjutkan penerimaan dan pemrosesan ratusan *request* HTTP lain secara konkuren.

Tantangan konkurensi muncul pada manajemen stateful. Untuk keperluan konversi tipe data ID Request berjenis *String* contoh: "*client-xyz*" ke *Integer* 8-byte yang dibutuhkan oleh *binary framing* pada protokol internal TCP/UD), saya mengimplementasikan mekanisme **Global Counter**. Untuk menjamin *Thread-Safe* dari objek global ini ketika diakses oleh ribuan *request* serentak, proses inkrementasi dilindungi oleh mekanisme penguncian sinkronisasi `threading.Lock()`. Hal ini mutlak diperlukan untuk mencegah insiden *Race Condition* yang dapat menghasilkan ID duplikat pada jaringan internal.

```
PS C:\Users\HP 14s\Downloads\towerofbabel> python -m pytest tests/smoke -v
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\HP 14s\AppData\Local\Programs\Python\Python313\python.exe
rootdir: C:\Users\HP 14s\Downloads\towerofbabel
configfile: pyproject.toml
plugins: anyio-4.14.2, Faker-38.2.0, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 3 items

tests/smoke/test_services_blackbox.py::test_service_a_http_contract PASSED [ 33%]
tests/smoke/test_services_blackbox.py::test_service_b_tcp_contract PASSED [ 66%]
tests/smoke/test_services_blackbox.py::test_service_c_udp_contract PASSED [100%]

3 passed in 0.25s
```

Gambar 2. Pembuktian validasi konkurensi, translasi protokol TCP, dan UDP melewati 100% integrasi Smoke Test

6. Penanganan Kegagalan

Ketahanan Gateway didasarkan pada prinsip arsitektural Fail-Fast dan Failure Isolation untuk mencegah fenomena *Cascading Failure* atau kegagalan beruntun yang meruntuhkan seluruh sistem.

- **Timeout Budget Management:** Gateway mengambil parameter `timeout_ms` dari kontrak klien dan mengonversinya menjadi batasan waktu mutlak pada sisi server. Setiap pemanggilan ke kelas *adapter* dibungkus rapat dalam blok `asyncio.wait_for`. Jika sebuah *backend* mengalami *delay* parah atau mogok (*hang*), tugas asinkron tersebut akan dibatalkan secara sepihak oleh Gateway begitu batas waktu terlewati. Gateway kemudian akan mengembalikan pesan galat semantik `BACKEND_TIMEOUT` dengan flag `retryable: true`, memastikan Service Level Agreement klien tetap terjaga tanpa harus menunggu selamanya.
- **Isolasi Kehancuran:** Interaksi jaringan dipenuhi oleh ketidakpastian. Segala bentuk *Exception* pada lapisan *transport* mulai dari paket CRC yang ditolak, koneksi TCP (*ConnectionRefusedError*), hingga format balasan JSON malformed dari *backend* ditangkap secara anggun melalui blok `try-except` global. Kematian satu layanan misalnya Service B lumpuh total, hanya akan menghasilkan pengembalian JSON *error* yang rapi pada *endpoint* klien terkait, tanpa membekukan atau menjatuhkan proses utama Uvicorn di Gateway. Layanan A dan C akan tetap beroperasi normal.

7. Keputusan dan Kompromi Teknis

Dalam merancang sistem ini, beberapa keputusan kompromi harus diambil untuk menyeimbangkan kompleksitas dengan keandalan:

- Memilih menyimpan konfigurasi fungsional secara *hardcoded* di memori RAM memberikan keuntungan latensi serendah mungkin dan kemudahan *deployment* tanpa *container* tambahan. Komprominya, arsitektur ini mengorbankan fleksibilitas; setiap penambahan kapabilitas atau layanan *backend* baru mewajibkan *developer* untuk memodifikasi kode sumber dan melakukan *restart* Gateway (bukan kapabilitas dinamis *runtime*).

- Saya menggunakan `DatagramProtocol` dengan paradigma *fire-and-forget* sederhana yang dibatasi oleh batas waktu. saya memutuskan untuk tidak mengimplementasikan algoritma antrean (*queueing*) atau pengiriman ulang (*re-transmission*) yang persisten di level Gateway. Komprominya, Gateway kehilangan kekokohan dalam menghadapi jaringan *lossy* yang sering menghilangkan paket. Tanggung jawab untuk mengeksekusi panggilan ulang (*retry logic*) sepenuhnya digeser kembali ke Klien Utama, yang difasilitasi oleh respons `retryable: true` dari Gateway.

8. Keterbatasan Implementasi

Sistem saat ini memiliki *blind spots* yang perlu ditangani untuk skenario *production* skala masif:

- Absennya Circuit Breaker & Rate Limiting: Gateway bertindak secara pasif dengan meneruskan seluruh *traffic* secara membabi buta. Jika terjadi serangan *flood request*, tidak ada algoritma *Token Bucket* yang membatasi laju masuk, sehingga server *backend* khususnya Service B sangat berpotensi kelebihan muatan. Begitu pula, jika *backend* sedang mati, Gateway akan terus mencoba mengirimkan permintaan alih-alih membuka *Circuit Breaker* untuk memberi waktu *backend* memulihkan diri.
- Tidak Ada Retry Internal Secara Mandiri: Limitasi implementasi tidak mencakup kapabilitas untuk mengulang transmisi jaringan yang gagal dari dalam tubuh Gateway itu sendiri. *Timeout budget* murni difungsikan sebagai jaring pengaman tunggal sebelum Gateway menyerah terhadap suatu *request*.

9. Invariansi Sistem Tambahan

Sebagai jaminan ketahanan level korporat, sistem ini berhasil menegakkan dua invariansi sistem ekstra yang krusial untuk kestabilan jangka panjang:

1. Resource Isolation: Setiap panggilan ke Service B membuka koneksi soket TCP. Penutupan soket secara bersih (`writer.close()` dan rutin asinkron `await writer.wait_closed()`) diimplementasikan secara mutlak di dalam blok `finally`. Pola ini memberikan jaminan 100% bahwa Sistem Operasi tidak akan kehabisan *file descriptors* mencegah kebocoran memori/sumber daya bahkan saat eksekusi digugurkan paksa oleh galat *TimeoutError* di pertengahan proses transmisi byte.
2. Correctness through Checksum Validation: Pada penerimaan respons UDP asinkron dari Service C, invariansi integritas data ditegakkan dengan ketat. Gateway tidak serta merta memercayai *payload* yang datang. Sistem memisahkan *trailer* 4-byte dan melakukan perhitungan rekursif ulang menggunakan `zlib.crc32` pada potongan *header* dan *payload*. Jika nilai *hash* kalkulasi Gateway berbeda dengan nilai byte *trailer*, paket akan langsung dijatuhkan secara diam-diam atau *silent drop* dari *Event*

Loop. Invariansi ini menjamin nol toleransi terhadap masuknya korupsi bit ke lapisan logika bisnis.